

Module 6 Assignment: Otomoto Marketing Segmentation Model Optimization

Emmanuel Fru
BAN6440 – Machine Learning
Professor: Dr. Preethi Natarajan
Nexford University

September 13, 2026

Project repository: <https://github.com/fruStoic/otomoto-ann-optimization>

1 Business Context and Problem Framing

The assignment presents the task as optimizing an artificial neural network used by Otomoto for marketing segmentation. However, the supplied dataset does not contain automotive customers or marketing segment labels. It contains 7,043 telecommunications customer records covering account tenure, subscribed services, contract type, billing method, monthly charges, total charges, and whether the customer churned.

For this reason, I treated the dataset as a supervised customer churn classification problem. This still supports the marketing objective of the assignment. Customers identified as having a higher probability of churn can be targeted with retention campaigns, while lower-risk customers can be handled differently. Churn prediction is widely used in telecommunications because identifying customers at risk of leaving can support more focused retention activity (Vafeiadis et al., 2015; Ahmad et al., 2019; Amin et al., 2019).

The target variable is moderately imbalanced. Of the 7,043 customers, 5,174 did not churn and 1,869 did. This means 73.46% of customers belong to the majority class. A model that predicted “No Churn” for every customer would therefore achieve 73.46% accuracy without learning anything useful. I used this as the minimum accuracy benchmark when evaluating the ANN.

The objective of the optimization was to determine whether optimizer choice could improve the performance of the ANN. Adam, RMSprop, and stochastic gradient descent (SGD) were compared while keeping the dataset, ANN architecture, preprocessing, batch size, loss function, and early-stopping rule fixed. Each optimizer was run across five random seeds to reduce the influence of a single favourable or unfavourable training run.

F1-score for the churn class was used as the primary selection metric, with recall as the first tie-

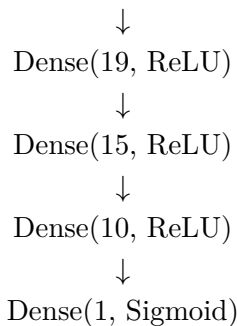
breaker. Recall matters because failing to identify a customer who is likely to churn can result in a missed retention opportunity. Precision also matters because incorrectly flagging too many customers would increase the cost of the marketing campaign.

2 Existing ANN and Baseline Assessment

I first recreated the ANN logic supplied with the assignment so that there was a clear “before optimization” result. The existing workflow label-encoded categorical variables, dropped the 11 rows where `TotalCharges` was blank, used a 75/25 train-test split, applied SMOTE to the training data, and trained an Adam-based neural network for five epochs. SMOTE creates synthetic minority-class observations to reduce class imbalance (Chawla et al., 2002).

The recreated network contained 19 input features followed by hidden layers of 19, 15, and 10 neurons, all using ReLU activation, and a single sigmoid output for binary classification. Its loss function was binary cross-entropy.

19 input features



The legacy model produced the results in Table 1.

Table 1: Recreated legacy ANN results

Metric	Result
Test binary cross-entropy loss	0.9022
Accuracy	56.9%
Precision	37.0%
Recall	88.7%
F1-score	52.2%
ROC-AUC	74.4%
True negatives	586
False positives	705
False negatives	53
True positives	414

The legacy model’s test binary cross-entropy loss was 0.9022. Its main strength was recall. It identified 414 of 467 churners in its test split. However, the overall model was not suitable for an

efficient retention campaign. Its 56.9% accuracy was below the 73.5% majority-class benchmark. Precision was only 37.0%, and 705 non-churners were incorrectly classified as churn risks. This means the model caught many genuine churners by also flagging a very large number of customers who were not going to leave.

The baseline also had methodological weaknesses. Label encoding gives categorical values arbitrary integer relationships that do not necessarily represent the meaning of the categories. The numeric variables were not standardised. The 11 blank **TotalCharges** records were removed even though all 11 belonged to customers with zero tenure. The supplied workflow also used the test split during training as validation data. I treated these issues as part of the model optimization problem rather than attributing all improvement to optimizer choice alone.

3 Data Preparation and Revised ANN

3.1 Data Cleaning

The raw dataset contained 7,043 rows and 21 columns. There were no duplicate rows and no duplicate customer IDs. Pandas initially reported no missing values, but inspection of **TotalCharges** identified 11 blank strings. All 11 occurred where **tenure** = 0. I converted **TotalCharges** to numeric and assigned these records a value of 0 rather than deleting the customers. This retained all 7,043 observations.

Two additional relationships were found during data inspection. Across 1,526 customers with **InternetService** = No, six internet-related service variables were perfectly aligned with that condition. This shows that several service fields carry overlapping information rather than independent signals.

I also found that **TotalCharges** was almost perfectly correlated with **tenure** multiplied by **MonthlyCharges**, with a correlation of 0.9996. This is consistent with **TotalCharges** acting largely as an accumulated billing measure. I retained the variable because the optimization task focused on the ANN rather than feature elimination, but the relationship should be considered in future feature-reduction experiments.

The **customerID** field was removed because it is an identifier rather than a predictive feature. The target variable was encoded as:

- Churn = Yes $\rightarrow$ 1
- Churn = No $\rightarrow$ 0

The remaining dataset contained 19 raw predictors.

3.2 Encoding, Scaling, and Data Split

The categorical predictors were transformed using one-hot encoding with the first category dropped and unknown categories ignored. One-hot encoding represents categories as binary indicator vari-

ables rather than imposing an artificial numerical order (Scikit-learn Developers, n.d.-a). After transformation, the ANN received 30 input features.

The numeric variables were:

- `SeniorCitizen`
- `tenure`
- `MonthlyCharges`
- `TotalCharges`

These variables operate on different scales, especially `TotalCharges` compared with the binary `SeniorCitizen` field. I therefore used `StandardScaler`. The scaler and encoder were fitted on the training split only, then applied to the validation and test sets. This avoids using held-out data statistics during training, which would create data leakage (Scikit-learn Developers, n.d.-b).

The cleaned data was split using a stratified 70/15/15 train-validation-test design with `random_state=42`.

Table 2: Final data split

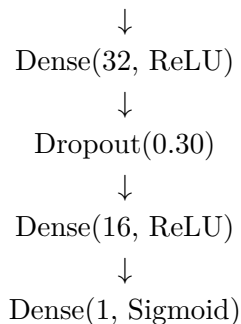
Split	Rows	Churn	Churn Rate
Training	4,930	1,308	26.53%
Validation	1,056	280	26.52%
Test	1,057	281	26.58%

The numeric training features had means effectively equal to zero and standard deviations effectively equal to one after scaling. The similar churn rates across all three splits confirmed that stratification worked as intended.

3.3 Revised ANN Architecture

For the controlled optimizer experiment, I used one fixed ANN architecture for all three optimizers:

30 encoded input features



The revised architecture is small enough for a dataset of this size while providing more capacity than the original model to work with the expanded one-hot encoded feature space. Dropout was included as regularisation. The loss function remained binary cross-entropy.

This means the before-and-after comparison is a comparison of the supplied legacy workflow against the complete revised pipeline. It should not be interpreted as evidence that optimizer choice alone produced the full performance difference. To isolate optimizer effects, Adam, RMSprop, and SGD were compared only within the revised pipeline using the same architecture and data split.

4 Optimization Methodology

4.1 Optimizer Selection

I compared three optimizers named directly in the assignment and commonly used for neural-network training:

- **Adam**, with learning rate 0.001
- **RMSprop**, with learning rate 0.001
- **SGD**, with learning rate 0.01

Adam adapts updates using estimates of the first and second moments of the gradients (Kingma and Ba, 2015; TensorFlow, n.d.-a). RMSprop scales updates using a moving average of squared gradients (TensorFlow, n.d.-c). SGD applies gradient-based weight updates directly and used the larger learning rate of 0.01 in this experiment (TensorFlow, n.d.-d).

The learning rates were declared explicitly rather than relying on hidden defaults. The larger SGD learning rate was intentional because plain SGD generally uses a larger update step than Adam and RMSprop. The goal was to compare reasonable optimizer configurations, not to force every algorithm to use the same numerical learning rate.

4.2 Controlled Experiment Design

All three optimizers used the same training, validation, and test split, ANN architecture, binary cross-entropy loss, batch size, dropout rate, classification threshold, maximum epoch budget, and early-stopping rule. This controlled most experimental conditions, but it did not guarantee that all optimizers reached the same convergence state within the 150-epoch budget. That distinction became important in the results because Adam and RMSprop early-stopped well before the cap, while SGD frequently required much longer training.

The common experiment settings were:

- batch size of 32;
- maximum of 150 epochs;
- dropout rate of 0.30;
- validation-loss early stopping with patience of 10;
- classification threshold of 0.50.

TensorFlow’s early stopping callback can stop training when the monitored validation metric stops improving and restore the best weights (TensorFlow, n.d.-b).

Neural-network training is stochastic, so a single training run can give a misleading comparison. I therefore trained every optimizer across five seeds: 42, 123, 456, 789, and 2026. This produced 15 validation experiments. The customer split remained fixed at `random_state=42`; only the network initialization and training randomness changed.

The final optimizer was selected using validation data only. The decision rule was defined before the test set was evaluated:

1. highest mean validation F1-score;
2. highest mean recall if F1 was tied;
3. highest mean precision if the first two metrics were tied.

This approach is consistent with prior churn research that evaluates multiple classification metrics rather than relying on accuracy alone (Vafeiadis et al., 2015; Amin et al., 2019).

5 Optimizer Comparison Results

Table 3 reports mean and standard deviation across the five seeds.

Table 3: Validation performance across five random seeds

Optimizer	Accuracy	Precision	Recall	F1	ROC-AUC	Epochs
Adam	.802 ± .006	.648 ± .024	.564 ± .027	.602 ± .008	.846 ± .002	31.8 ± 5.3
RMSprop	.801 ± .006	.647 ± .021	.550 ± .027	.594 ± .012	.843 ± .001	33.6 ± 7.8
SGD	.804 ± .002	.648 ± .010	.576 ± .032	.609 ± .014	.845 ± .001	97.6 ± 32.4

SGD achieved the highest mean validation F1-score at 0.609 and the highest mean recall at 0.576. It was therefore selected under the predefined decision rule. However, the result needs qualification. SGD’s mean F1 advantage over Adam was only 0.007, which is small relative to the observed run-to-run variation. Adam also converged much faster, averaging 31.8 epochs compared with 97.6 for SGD, and its F1-score was more stable across seeds.

RMSprop did not provide a clear advantage. Its mean F1, recall, accuracy, and ROC-AUC were slightly below the other two configurations.

The optimizer comparison therefore produced two different conclusions. SGD was the nominal performance winner under the predefined rule. Adam was the more computationally efficient alternative. The difference between them was not large enough to argue that SGD is universally superior for this dataset.

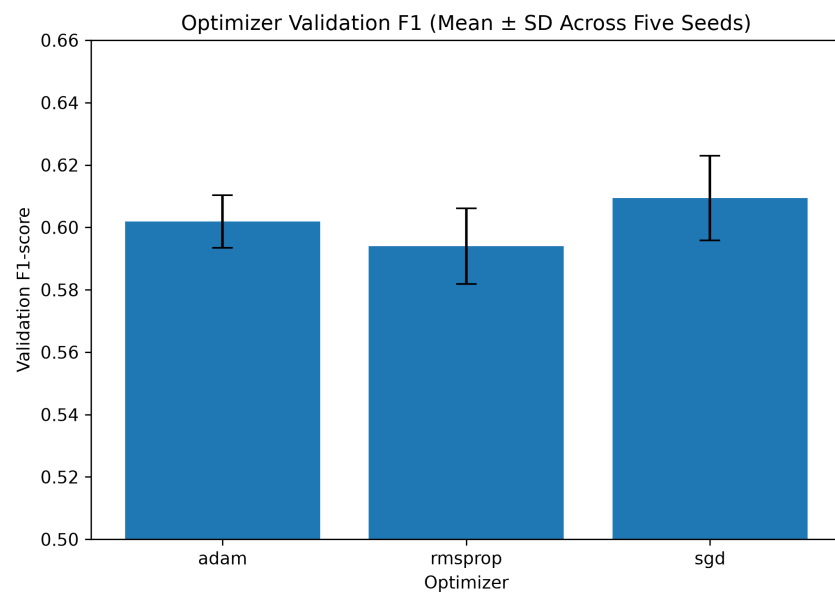


Figure 1: Validation F1-score by optimizer, reported as mean and standard deviation across five seeds.

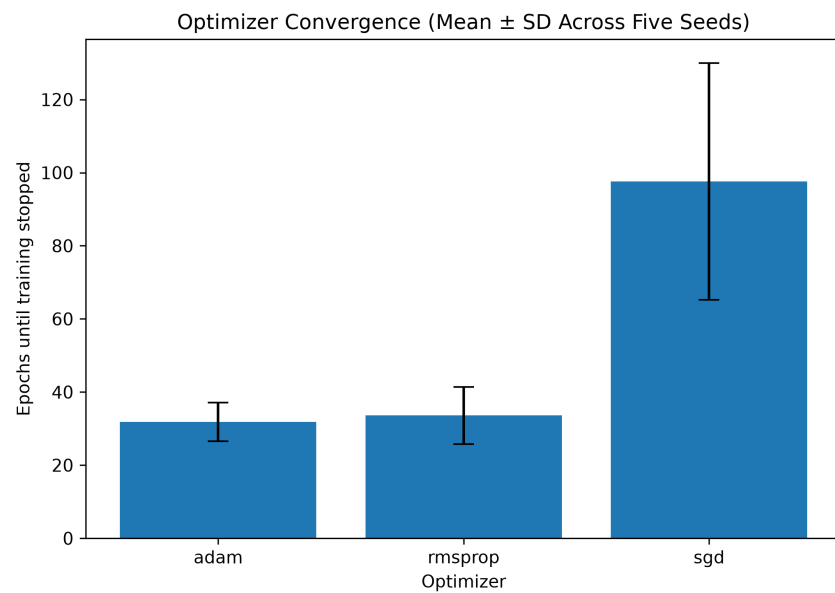


Figure 2: Average number of training epochs by optimizer.

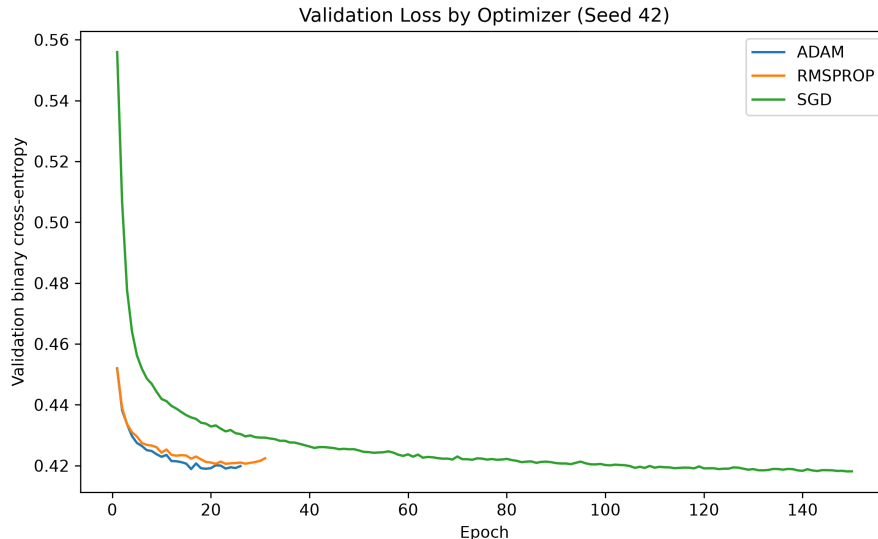


Figure 3: Representative validation-loss curves for seed 42. The seed was fixed before examining the results.

6 Final Model Evaluation

After SGD was selected from validation results, I trained the fixed SGD configuration using seed 42 and evaluated the 1,057-case test set once. I did not change the optimizer, architecture, learning rate, threshold, or seed after seeing the test results.

The final training run reached the 150-epoch maximum, and the best validation-loss epoch was 149. Early stopping therefore did not terminate training before the epoch cap. This suggests that SGD convergence was still incomplete under the selected configuration. Because the test set had already been evaluated, I did not increase the epoch limit and retune the final model afterward.

Table 4: Final held-out SGD test performance

Metric	Result
Test binary cross-entropy loss	0.4175
Accuracy	80.7%
Precision	69.0%
Recall	49.8%
F1-score	57.9%
ROC-AUC	84.5%
True negatives	713
False positives	63
False negatives	141
True positives	140

The final model’s held-out binary cross-entropy loss was 0.4175. The majority-class accuracy on the test set was 73.4%. The ANN therefore improved accuracy by 7.3 percentage points while also

producing meaningful discrimination between churners and non-churners, with ROC-AUC of 0.845.

Of the 281 customers who actually churned, the model identified 140 and missed 141. Of the 776 customers who did not churn, only 63 were incorrectly flagged. The model predicted 203 customers as churn risks in total, which is 19.2% of the test population. Of those 203 customers, 69.0% were actual churners. This is much more selective than the legacy model and would allow a marketing team to focus retention activity on a smaller group.

The final metrics should still be treated as estimates. The 95% Wilson confidence intervals were:

- Accuracy: 78.2% to 83.0%
- Precision: 62.3% to 74.9%
- Recall: 44.0% to 55.6%

The recall interval is especially important. The model does not identify every customer who will churn, and the point estimate should not be treated as a fixed population value.

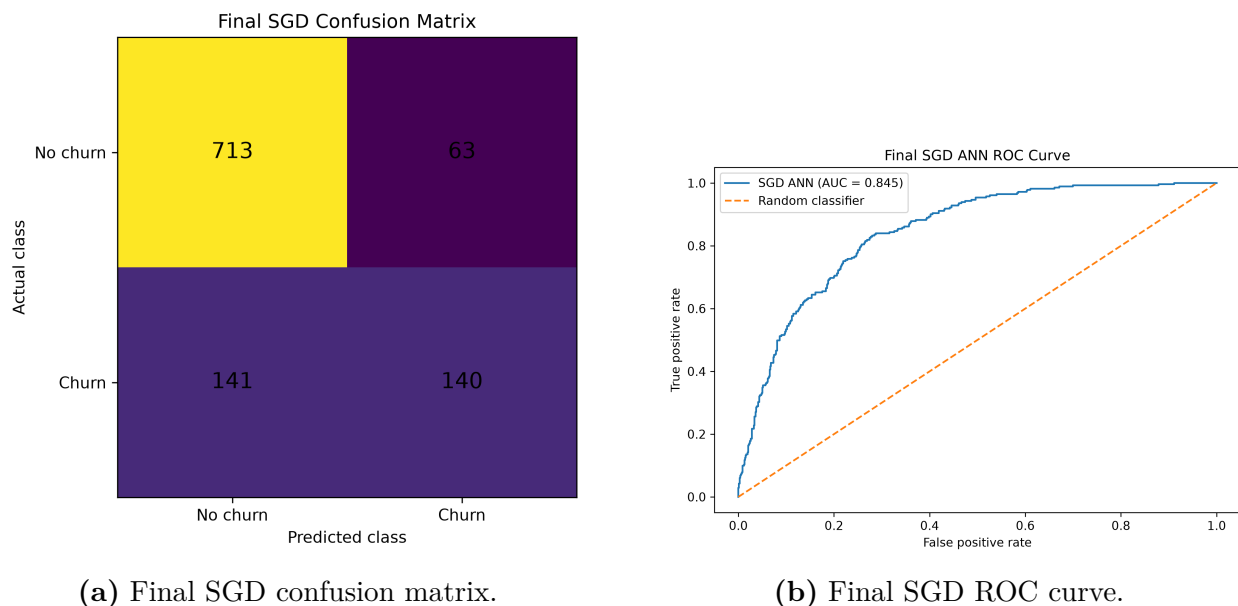


Figure 4: Final held-out test diagnostics.

7 Before and After Optimization

Table 5 compares the recreated legacy model with the final revised model.

Table 5: Legacy model and final revised model

Metric	Legacy ANN	Final SGD ANN
Test binary cross-entropy loss	0.9022	0.4175
Accuracy	56.9%	80.7%
Precision	37.0%	69.0%
Recall	88.7%	49.8%
F1-score	52.2%	57.9%
ROC-AUC	74.4%	84.5%

The revised model improved accuracy, precision, F1-score, and ROC-AUC substantially. Its test binary cross-entropy loss was also lower, decreasing from 0.9022 in the legacy workflow to 0.4175 in the revised workflow. Recall decreased. This means the final model became more selective, but it also missed more customers who eventually churned. Because the two models were evaluated on different samples and under different protocols, the loss difference should also be interpreted as a workflow-level comparison rather than a controlled paired estimate.

The legacy ANN was optimized heavily toward identifying the positive class because SMOTE balanced the training set and the model produced many positive predictions. That gave it high recall but only 37.0% precision. A retention team using that model would contact many customers who were not actually likely to churn.

The revised model reduced this problem. Precision increased to 69.0%, so a larger proportion of the customers flagged for intervention were genuine churners. The trade-off is that recall fell to 49.8%. Whether that trade-off is appropriate depends on the campaign economics. If contacting customers is expensive, higher precision can be valuable. If the cost of missing a churner is much greater than the cost of an unnecessary retention offer, the decision threshold should be calibrated toward higher recall before deployment.

The two rows in Table 5 were produced under different evaluation protocols and were also measured on non-identical samples. The legacy model was evaluated on 1,758 observations from a 75/25 split, while the revised model was evaluated on 1,057 observations from the fixed 15% test split. The two evaluation samples are not directly matched, and some observations used for legacy testing may belong to the revised training set. The table should therefore be interpreted as a before-and-after workflow comparison rather than a controlled paired model comparison.

The legacy workflow also used SMOTE, label encoding, no numeric scaling, and a different network architecture. The controlled optimizer experiment in Table 3 is the more appropriate evidence for comparing Adam, RMSprop, and SGD. One of the main findings from the assignment is that data preparation and evaluation design produced larger practical differences than the small performance gap between the leading optimizers.

8 Marketing Implications and Recommendations

The final ANN can be used as a risk-ranking tool for retention campaigns rather than as a fully automated decision system. At the default 0.50 threshold, the model flagged 203 of 1,057 test customers, or 19.2%, as churn risks. This concentrates marketing resources on a much smaller group than contacting the full customer base.

The 69.0% precision means the flagged group is relatively focused. However, recall of 49.8% means roughly half of the actual churners were not identified at the selected threshold. For that reason, I would not treat 0.50 as a permanent business threshold. The validation analysis suggests that a lower threshold, particularly around 0.35, may provide a better balance for retention marketing by recovering more potential churners while keeping precision above 50%.

8.1 Validation-Based Threshold Analysis

The final test model was evaluated at the pre-specified classification threshold of 0.50. I did not change that threshold after observing the test results. However, I conducted a separate threshold analysis using validation data only to identify a more useful operating range for a future retention campaign.

Table 6: SGD validation performance across classification thresholds

Threshold	Precision	Recall	F1-score
0.30	.522 $\pm$.012	.788 $\pm$.012	.628 $\pm$.008
0.35	.545 $\pm$.009	.740 $\pm$.015	.627 $\pm$.006
0.40	.572 $\pm$.009	.693 $\pm$.022	.626 $\pm$.006
0.45	.606 $\pm$.011	.641 $\pm$.024	.623 $\pm$.006
0.50	.648 $\pm$.010	.576 $\pm$.032	.609 $\pm$.014

The threshold sweep shows that the default 0.50 cutoff is not necessarily the best operating point for a retention campaign. Lowering the threshold to 0.30 produced the highest mean validation F1-score at 0.628 and increased recall from 57.6% to 78.8%. The trade-off was lower precision, which fell from 64.8% to 52.2%.

A threshold of 0.35 provides a practical middle ground. It achieved 74.0% recall, 54.5% precision, and an F1-score of 0.627. Its F1-score was almost identical to the maximum observed at 0.30, while producing fewer false positive predictions.

I would therefore recommend 0.35 as a provisional operating threshold for a retention campaign where missing likely churners is more costly than contacting some customers who would not have churned. This recommendation should still be validated against the actual cost of a retention intervention and the expected value of retaining a customer before deployment.

The held-out test set was not used in this threshold analysis, and the final test metrics reported earlier remain the official test results at the original 0.50 threshold.

8.2 Operational Recommendations

I would recommend the following next steps:

1. Re-run the optimizer comparison with a higher epoch cap so that SGD, Adam, and RMSprop can all reach a stable stopping point before being compared.
2. Add a minimum-effect-size rule to optimizer selection. If performance differences fall within the observed run-to-run spread, prefer the cheaper or faster configuration.
3. Use approximately 0.35 as a provisional retention threshold, then adjust it using real campaign economics.
4. Compare the selected ANN with simpler baselines such as logistic regression and tree-based models. Previous churn studies show that different algorithms can perform well depending on the data and evaluation setup (Vafeiadis et al., 2015; Ahmad et al., 2019).
5. Monitor precision, recall, churn prevalence, and calibration after deployment. Customer behaviour can change over time.
6. Retrain only when monitoring shows meaningful deterioration or when the customer population, products, or pricing structure changes.

9 Challenges, Limitations, and Ethical Considerations

9.1 Challenges and Lessons Learned

The first challenge was that the assignment framing did not match the supplied data. The brief described Otomoto marketing segmentation, while the file contained labelled telecom churn data. I resolved this by treating churn probability as the basis for marketing segmentation instead of forcing an unsupervised segmentation method onto a supervised target.

The second challenge was hidden data quality. `TotalCharges` appeared complete until the column was inspected as text. Eleven records contained blank strings rather than `NaN`. All 11 belonged to customers with zero tenure, so I converted them to zero instead of deleting them.

A third lesson came from the optimizer experiment. SGD had the best mean F1-score, but its advantage over Adam was small relative to the variation across seeds. A single run could easily have produced a different ranking. Running five seeds made that uncertainty visible.

The pre-defined selection rule also had no minimum-effect-size condition. SGD’s mean F1 advantage over Adam was only 0.007, while SGD’s own F1 standard deviation was 0.014. I followed the rule as specified, but a stronger rule would treat differences inside the observed run-to-run spread as practically tied and then prefer the less expensive optimizer.

The final SGD run also reached the 150-epoch cap with the best validation-loss epoch at 149. This means the early-stopping condition did not actually stop training. The result is still valid as the

preselected final experiment, but it shows that the convergence settings were not sufficient for an equal optimizer comparison.

9.2 Methodological Limitations

The final model has several limitations.

First, optimizer selection used five seeds but only one fixed train-validation split. Repeated stratified cross-validation would provide stronger evidence about performance across different customer samples.

Second, a further limitation is that the three optimizers shared an epoch budget but not a convergence state. Adam and RMSprop early-stopped well inside the 150-epoch cap, while SGD did not. The comparison in Table 3 therefore contrasts two configurations that had effectively converged with one that was still improving. Holding the training budget fixed is not equivalent to holding convergence fixed. A stronger design would either raise the epoch cap until every optimizer early-stops naturally or evaluate each optimizer at its own stable convergence point.

Third, the final model was evaluated using one threshold of 0.50. The threshold was intentionally not changed after seeing the test set. The additional validation-only threshold analysis suggests that 0.35 may provide a better retention operating point, but that recommendation still requires business cost inputs before deployment.

Fourth, the legacy and final models used different preprocessing and evaluation procedures. This means the large before-and-after improvement cannot be assigned to optimizer choice alone.

Fifth, the final test recall was 49.8%, with a 95% interval of 44.0% to 55.6%. A retention process relying only on the default 0.50 threshold would miss many genuine churners.

9.3 Ethical Application of the Model

Churn predictions should support marketing decisions, not automatically determine how individual customers are treated. Variables such as age-related status, contract type, payment method, and service usage can create different error rates across customer groups. Before deployment, model performance should therefore be checked across relevant customer segments rather than assuming that the aggregate metrics apply equally to everyone.

The data should also be limited to information needed for the retention purpose. Customer predictions should not be used to deny service, impose penalties, or target customers in ways that create unfair treatment. Human review remains important when a prediction leads to a meaningful customer action.

A practical ethical concern is also the balance between false positives and false negatives. High recall is not automatically better if it creates intrusive or wasteful contact with large numbers of customers. High precision is not automatically better if it ignores many customers at genuine risk. The threshold should reflect a documented business objective and should be reviewed as customer behaviour changes.

9.4 Responsible Use of AI During the Assignment

AI tools were used during the project for research support, code generation, debugging, experiment design, result interpretation, and report development. I did not treat generated output as evidence by itself. Code was executed locally, errors were corrected from observed behaviour, and reported metrics were taken from saved experiment outputs.

The interaction with AI also changed during the project. Initial suggestions were revised when they did not fit the dataset or experimental objective. For example, the supplied assignment described marketing segmentation, but the data supported supervised churn classification. The optimizer experiment was also expanded from a possible single-run comparison to five seeds so that random training variation could be observed.

AI-generated recommendations were also challenged after the results were available. SGD technically won under the pre-defined F1 selection rule, but its small advantage over Adam and slower convergence showed that the result needed qualification. This review process was important because simply accepting the highest metric would have overstated the evidence.

The required AI Disclosure Form provides the full record of the tools used, their roles, prompt refinement, verification steps, and modifications made after review.

10 Software Validation and Code Organization

The project was organized into separate modules for loading data, preprocessing, model construction, evaluation, optimizer experiments, result analysis, threshold analysis, and final evaluation. This kept the training logic separate from data preparation and reduced the risk of changing multiple parts of the pipeline during an optimizer comparison.

The automated test suite contained 11 tests. All 11 passed in 6.45 seconds. The tests covered:

- removal of the customer identifier;
- conversion of blank `TotalCharges` to zero;
- removal of remaining missing values;
- correct churn target encoding;
- consistent encoded feature dimensions across train, validation, and test;
- correct numeric scaling;
- preservation of class balance through stratification;
- successful construction of Adam, RMSprop, and SGD;
- valid ANN probability output.

The successful pytest log was saved to `outputs/pytest_results.txt`. The optimizer experiment results, histories, summaries, threshold results, charts, and final test metrics were also saved under

outputs/. This makes every number reported in the analysis traceable to a project output rather than to memory.

Code and Data Availability

The complete Python implementation, supplied dataset, automated test suite, experiment outputs, and report source are available at <https://github.com/fruStoic/otomoto-ann-optimization>.

References

- Ahmad, A. K., Jafar, A., & Aljoumaa, K. (2019). *Customer churn prediction in telecom using machine learning in big data platform*. Journal of Big Data, 6, 28. <https://doi.org/10.1186/s40537-019-0191-6>
- Amin, A., Al-Obeidat, F., Shah, B., Adnan, A., Loo, J., & Anwar, S. (2019). *Customer churn prediction in telecommunication industry using data certainty*. Journal of Business Research, 94, 290–301. <https://doi.org/10.1016/j.jbusres.2018.03.003>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. International Conference on Learning Representations. <https://arxiv.org/abs/1412.6980>
- Scikit-learn Developers. (n.d.-a). *OneHotEncoder*. Scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
- Scikit-learn Developers. (n.d.-b). *StandardScaler and feature scaling guidance*. Scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- TensorFlow. (n.d.-a). *tf.keras.optimizers.Adam*. TensorFlow API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam
- TensorFlow. (n.d.-b). *tf.keras.callbacks.EarlyStopping*. TensorFlow API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping
- TensorFlow. (n.d.-c). *tf.keras.optimizers.RMSprop*. TensorFlow API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/RMSprop
- TensorFlow. (n.d.-d). *tf.keras.optimizers.SGD*. TensorFlow API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/SGD
- Vafeiadis, T., Diamantaras, K. I., Sarigiannidis, G., & Chatzisavvas, K. C. (2015). *A comparison of machine learning techniques for customer churn prediction*. Simulation Modelling Practice and Theory, 55, 1–9. <https://doi.org/10.1016/j.simpat.2015.03.003>